3. Manual de la librería C# — LibC#

3.1. Descripción

`LibC#` es una librería **cliente ADS** para .NET 6.0, sin paquetes NuGet externos (solo la BCL). Expone el espacio de nombres `Beckhoff.Ads` y permite conectar, leer, escribir y suscribir notificaciones contra un dispositivo ADS (real o el simulador Flutter).

3.2. Compilación

```
cd LibC#
dotnet build src/Beckhoff.Ads/Beckhoff.Ads.csproj
```

3.3. Uso rápido

```
using Beckhoff.Ads;

using var client = new TcAdsClient(
    amsNetId: "127.0.0.1.1.1",
    amsPort: AdsPorts.PlcTc3,
    ipAddress: "127.0.0.1",
    tcpPort: AdsPorts.TcpPort);

client.Connect();

var (state, deviceState) = client.ReadState();
ushort counter = client.ReadValue<ushort>("MAIN.nCounter", "UINT");
client.WriteValue<bool>("MAIN.bStart", true, "BOOL");
float temp = client.ReadValue<float>("MAIN.fTemp", "REAL");
```

3.4. Referencia de clases

AmsNetId (struct)

NetId AMS de 6 bytes. Métodos: `Parse(string)`, `ToByteArray()`, `FromByteArray(byte[], offset)`, `ToString()`, `IsEmpty`, `Empty`, operadores `==`/`!=`.

AmsAddress (struct)

Dirección AMS (`NetId` + `Port`).

AmsTcpHeader (struct)

Cabecera AMS/TCP de 6 bytes. `Write(Span<byte>)` y `Read(ReadOnlySpan<byte>)`.

AmsHeader (class)

Cabecera AMS de 32 bytes. Propiedades: `TargetNetId`, `TargetPort`,

`SourceNetId`, `SourcePort`, `CommandId`, `StateFlags`, `Length`, `ErrorCode`,

`InvokeId`. Métodos `Write`/`Read`.

AdsConstants / AdsCommands / AdsIndexGroups / AdsPorts

Constantes del protocolo (comandos, grupos de índice y puertos).

AdsState (enum) y AdsTransMode (enum)

Estados del dispositivo y modos de transmisión de notificación.

AdsErrorCode (static class)

Códigos de error (sección 8 del PROTOCOL) y `GetMessage(uint)`.

AdsException : Exception

Excepción con propiedad `ErrorCode`.

AdsDeviceInfo

Resultado de `ReadDeviceInfo()`: `MajorVersion`, `MinorVersion`, `BuildVersion`,

`DeviceName`.

AdsSymbolInfo

Resultado de `GetSymbolInfo(name)`: `IndexGroup`, `IndexOffset`, `Size`,

`DataType`, `Flags`, `Name`, `TypeName`, `Comment`.

NotificationStamp

Muestra de notificación: `Handle`, `Timestamp`, `Data`.

PlcType (static class)

Codificación/decodificación de tipos PLC. Métodos: `SizeOf(plcType)`,

`Encode(value, plcType)`, `Decode(data, plcType)`, `Decode<T>(data, plcType)`.

TcAdsClient : IDisposable

Cliente principal. Constructor:

```
new TcAdsClient(amsNetId, amsPort, ipAddress, tcpPort, timeoutMs)
```

Método	Qué hace
`Connect()` / `Disconnect()` / `Dispose()`	Gestión de la conexión TCP.

`ReadDeviceInfo()`	`READ_DEVICE_INFO` (0x0001).
`ReadState()`	`READ_STATE` (0x0004) → `(AdsState, ushort)`.
`WriteControl(state, deviceState, data)`	`WRITE_CONTROL` (0x0005).
`ReadAny(ig, io, length)` / `WriteAny(ig, io, data)`	Lectura/escritura por índice/offset (0x0002/0x0003).
`ReadWrite(ig, io, readLength, writeData)`	Comando `READ_WRITE` (0x0009).
`ReadSymbol(name[, size	plcType])`
`WriteSymbol(name, value)`	Escritura por nombre.
`CreateVariableHandle(name)` / `DeleteVariableHandle(h)`	Handles (0xF003 / 0xF014).
`ReadByHandle(h, length)` / `WriteByHandle(h, data)`	Acceso por handle (0xF004).
`GetSymbolInfo(name)`	Info de símbolo (0xF006).
`SumRead(entries)` / `SumWrite(entries)`	Comandos sum (0xF080 / 0xF081).
`AddDeviceNotification(ig, io, cbLength, transMode, maxDelay, cycleTime, callback)`	Suscribe notificación (0x0006).
`DeleteDeviceNotification(handle)`	Cancela notificación (0x0007).
`ReadValue<T>(name[, plcType])` / `WriteValue<T>(name, value[, plcType])`	Acceso tipado (infiere el tipo PLC desde `T`).
`ReadStruct<T>(name)` / `WriteStruct<T>(name, value)`	Structs `[StructLayout(Sequential, Pack=1)]`.

3.5. Estructura de archivos

Archivo	Qué hace
`AmsNetId.cs`, `AmsAddress.cs`	Tipos de dirección AMS.
`AmsTcpHeader.cs`, `AmsHeader.cs`	Cabeceras del protocolo.
`AdsConstants.cs`, `AdsErrorCode.cs`	Constantes y códigos de error.
`AdsException.cs`, `AdsDeviceInfo.cs`, `AdsSymbolInfo.cs`, `NotificationStamp.cs`	Tipos de datos.
`PlcType.cs`	Codificación de tipos PLC.
`PlcStruct.cs`	`PlcMember` y `PlcStruct` para leer/escribir estructuras completas.
`TcAdsClient.cs`	Cliente principal.

3.6. Pruebas

```
cd LibC#/tests/Ads.Tests
dotnet run
```

3.7. Leer/escribir estructuras completas

Para leer o escribir una **estructura completa** (todos sus miembros de una vez),

define su disposición con `PlcMember` y `PlcStruct`:

```
using Beckhoff.Ads;

var posicion = new PlcStruct(new[]
{
    new PlcMember("X", "LREAL", 0),      // offset 0 bytes
    new PlcMember("Y", "LREAL", 8),      // offset 8 bytes
    new PlcMember("Activo", "BOOL", 16), // offset 16 bytes
}, totalSize: 24);

using var client = new TcAdsClient(...);
client.Connect();

// Leer toda la estructura de golpe (una sola lectura)
Dictionary<string, object?> valores = client.ReadStruct("GVL.Posicion", posicion);

// Escribir toda la estructura de golpe (una sola escritura)
client.WriteStruct("GVL.Posicion", posicion, new Dictionary<string, object?>
{
    ["X"] = 3.0,
    ["Y"] = 4.0,
    ["Activo"] = false,
});
```

Elemento	**Qué hace**
`new PlcMember(nombre, tipoPlc, offsetBytes)`	Define un miembro (nombre, tipo y offset en bytes).
`new PlcStruct(miembros, totalSize)`	Define la estructura. Sin `totalSize`, se calcula como `max(offset + tamaño)`.
`client.ReadStruct(nombre, def)`	Lee la estructura y devuelve `Dictionary<string, object?>`.
`client.WriteStruct(nombre, def, valores)`	Escribe la estructura desde `Dictionary<string, object?>`.

> El `offset` de cada miembro es su **offset en bytes** dentro de la estructura.

> En el `.tpy` de TwinCAT viene como `BitOffs`: divídelo entre 8.